```
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report)

CSV_PATH = "weather.csv"
TEMP_COL = "temperature"
HUM_COL = "humidity"
PRES_COL = "pressure"
WIND_COL = "wind"
RAIN_COL = "rain"
N_STATES = 3
WINDOW = 7
RANDOM_STATE = 42
STATE_NAMES = ["Sunny", "Cloudy", "Rainy"]


def make_synthetic(n=1000):
    rng = np.random.default_rng(RANDOM_STATE)
    trans = np.array([[0.70, 0.25, 0.05],
                      [0.25, 0.45, 0.30],
                      [0.10, 0.35, 0.55]])
    means = {0: (32, 40, 1014, 8), 1: (28, 65, 1010, 12), 2: (24, 88, 1004, 18)}
    stds = (2.5, 8, 2.5, 4)
    state = 0
    rows = []
    for _ in range(n):
        state = rng.choice(3, p=trans[state])
        m = means[state]
        t, h, p, w = [rng.normal(m[i], stds[i]) for i in range(4)]
        h = float(np.clip(h, 10, 100))
        w = max(w, 0)
        rain = 1 if (state == 2 and rng.random() < 0.9) or (state == 1 and rng.random() < 0.1) else 0
        rows.append([t, h, p, w, rain])
    df = pd.DataFrame(rows, columns=[TEMP_COL, HUM_COL, PRES_COL, WIND_COL, RAIN_COL])
    for c in [TEMP_COL, HUM_COL]:
        df.loc[rng.choice(n, 15, replace=False), c] = np.nan
    return df


def load_data():
    if os.path.exists(CSV_PATH):
        df = pd.read_csv(CSV_PATH)
        print("Loaded dataset:", CSV_PATH)
    else:
        df = make_synthetic()
        print("weather.csv not found, using generated sample data")
    df = df[[TEMP_COL, HUM_COL, PRES_COL, WIND_COL, RAIN_COL]].copy()
    rain_map = {"yes": 1, "no": 0, "1": 1, "0": 0, "1.0": 1, "0.0": 0, "true": 1, "false": 0}
    df[RAIN_COL] = df[RAIN_COL].astype(str).str.strip().str.lower().map(rain_map)
    df = df.dropna(subset=[RAIN_COL]).reset_index(drop=True)
    for c in [TEMP_COL, HUM_COL, PRES_COL, WIND_COL]:
        df[c] = pd.to_numeric(df[c], errors="coerce")
        df[c] = df[c].fillna(df[c].median())
    df[RAIN_COL] = df[RAIN_COL].astype(int)
    return df


def find_hidden_states(df):
    cols = [TEMP_COL, HUM_COL, PRES_COL, WIND_COL]
    scaler = StandardScaler()
    scaled = scaler.fit_transform(df[cols])
    km = KMeans(n_clusters=N_STATES, n_init=10, random_state=RANDOM_STATE)
    raw = km.fit_predict(scaled)
    hum_mean = pd.Series(df[HUM_COL].values).groupby(raw).mean()
    order = hum_mean.sort_values().index.tolist()
    remap = {old: new for new, old in enumerate(order)}
    states = np.array([remap[c] for c in raw])
    return states, km, scaler, remap


def make_observations(df):
```

```
    hum_level = pd.qcut(df[HUM_COL].rank(method="first"), 3, labels=False).astype(int)
    temp_level = pd.qcut(df[TEMP_COL].rank(method="first"), 3, labels=False).astype(int)
    return (hum_level * 3 + temp_level).values


def estimate_hmm(states, obs, n_obs=9):
    pi = np.ones(N_STATES)
    for i in range(0, len(states), WINDOW):
        pi[states[i]] += 1
    pi = pi / pi.sum()
    A = np.ones((N_STATES, N_STATES))
    for t in range(len(states) - 1):
        A[states[t], states[t + 1]] += 1
    A = A / A.sum(axis=1, keepdims=True)
    B = np.ones((N_STATES, n_obs))
    for s, o in zip(states, obs):
        B[s, o] += 1
    B = B / B.sum(axis=1, keepdims=True)
    return pi, A, B


def forward_log(obs_seq, A, B, pi):
    alpha = pi * B[:, obs_seq[0]]
    scale = alpha.sum()
    alpha = alpha / scale
    log_prob = np.log(scale)
    for t in range(1, len(obs_seq)):
        alpha = (alpha @ A) * B[:, obs_seq[t]]
        scale = alpha.sum()
        alpha = alpha / scale
        log_prob += np.log(scale)
    return log_prob


def viterbi_log(obs_seq, A, B, pi):
    T = len(obs_seq)
    logA, logB, logpi = np.log(A), np.log(B), np.log(pi)
    delta = np.zeros((T, N_STATES))
    psi = np.zeros((T, N_STATES), dtype=int)
    delta[0] = logpi + logB[:, obs_seq[0]]
    for t in range(1, T):
        for j in range(N_STATES):
            scores = delta[t - 1] + logA[:, j]
            psi[t, j] = np.argmax(scores)
            delta[t, j] = np.max(scores) + logB[j, obs_seq[t]]
    best_last = int(np.argmax(delta[T - 1]))
    path = [best_last]
    for t in range(T - 1, 0, -1):
        best_last = psi[t, best_last]
        path.insert(0, int(best_last))
    return float(np.max(delta[T - 1])), path


def run_hmm(states, obs, A, B, pi):
    n = len(obs)
    window_ll = []
    window_start = []
    correct = 0
    total = 0
    decoded_all = np.zeros(n, dtype=int)
    for start in range(0, n - WINDOW + 1, WINDOW):
        seq = obs[start:start + WINDOW]
        ll = forward_log(seq, A, B, pi)
        _, path = viterbi_log(seq, A, B, pi)
        decoded_all[start:start + WINDOW] = path
        correct += int(np.sum(np.array(path) == states[start:start + WINDOW]))
        total += WINDOW
        window_ll.append(ll)
        window_start.append(start)
    window_ll = np.array(window_ll)
    threshold = np.percentile(window_ll, 5)
    unusual = [(window_start[i], round(float(window_ll[i]), 2)) for i in range(len(window_ll)) if window_ll[i] <= threshol
    return window_ll, window_start, threshold, unusual, correct / total, decoded_all


def rain_models(df):
    cols = [TEMP_COL, HUM_COL, PRES_COL, WIND_COL]
    data = df.copy()
    data["rain_tomorrow"] = data[RAIN_COL].shift(-1)
    data = data.dropna(subset=["rain_tomorrow"])
    X = data[cols]
    y = data["rain_tomorrow"].astype(int)
```

```
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y)
    scaler = StandardScaler()
    X_train_s = scaler.fit_transform(X_train)
    X_test_s = scaler.transform(X_test)
    models = {
        "Logistic Regression": LogisticRegression(max_iter=1000),
        "Decision Tree": DecisionTreeClassifier(max_depth=3, random_state=RANDOM_STATE),
        "AdaBoost": AdaBoostClassifier(
            estimator=DecisionTreeClassifier(max_depth=1),
            n_estimators=100, learning_rate=0.8, random_state=RANDOM_STATE),
    }
    results = []
    for name, model in models.items():
        model.fit(X_train_s, y_train)
        pred = model.predict(X_test_s)
        results.append({
            "Model": name,
            "Accuracy": accuracy_score(y_test, pred),
            "Precision": precision_score(y_test, pred, zero_division=0),
            "Recall": recall_score(y_test, pred, zero_division=0),
            "F1": f1_score(y_test, pred, zero_division=0),
        })
        print("\n=====", name, "=====")
        print("Confusion Matrix:")
        print(confusion_matrix(y_test, pred))
        print(classification_report(y_test, pred, zero_division=0))
    return pd.DataFrame(results), models, scaler


def plot_clusters(df, states):
    plt.figure(figsize=(7, 5))
    for s in range(N_STATES):
        m = states == s
        plt.scatter(df.loc[m, HUM_COL], df.loc[m, TEMP_COL], s=15, label=STATE_NAMES[s])
    plt.xlabel("Humidity")
    plt.ylabel("Temperature")
    plt.title("K-Means Weather Types (Hidden States)")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()


def plot_matrices(A, B):
    fig, ax = plt.subplots(1, 2, figsize=(13, 4))
    ax[0].imshow(A, cmap="Blues")
    ax[0].set_title("Transition Matrix A")
    ax[0].set_xticks(range(N_STATES))
    ax[0].set_yticks(range(N_STATES))
    ax[0].set_xticklabels(STATE_NAMES)
    ax[0].set_yticklabels(STATE_NAMES)
    for i in range(N_STATES):
        for j in range(N_STATES):
            ax[0].text(j, i, f"{A[i, j]:.2f}", ha="center", va="center")
    ax[1].imshow(B, cmap="Oranges")
    ax[1].set_title("Emission Matrix B")
    ax[1].set_yticks(range(N_STATES))
    ax[1].set_yticklabels(STATE_NAMES)
    ax[1].set_xticks(range(B.shape[1]))
    ax[1].set_xlabel("Observation code (humidity level x 3 + temperature level)")
    for i in range(N_STATES):
        for j in range(B.shape[1]):
            ax[1].text(j, i, f"{B[i, j]:.2f}", ha="center", va="center", fontsize=8)
    plt.tight_layout()


def plot_decoded(states, decoded, days=42):
    plt.figure(figsize=(10, 4))
    plt.step(range(days), states[:days], where="mid", label="Actual (K-Means)", linewidth=2)
    plt.step(range(days), decoded[:days], where="mid", label="Viterbi Decoded", linestyle="--", linewidth=2)
    plt.yticks(range(N_STATES), STATE_NAMES)
    plt.xlabel("Day")
    plt.title("Actual vs Viterbi Decoded Weather")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()


def plot_likelihood(window_ll, threshold):
    plt.figure(figsize=(10, 4))
    plt.plot(window_ll, marker="o", markersize=3)
    plt.axhline(threshold, color="red", linestyle="--", label="Unusual week threshold")
```

```
    plt.xlabel("Week number")
    plt.ylabel("Forward log-probability")
    plt.title("Forward Algorithm: Likelihood of Each Week")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()


def plot_models(results):
    metrics = ["Accuracy", "Precision", "Recall", "F1"]
    x = np.arange(len(results))
    width = 0.2
    plt.figure(figsize=(9, 5))
    for i, m in enumerate(metrics):
        plt.bar(x + i * width, results[m], width, label=m)
    plt.xticks(x + 1.5 * width, results["Model"])
    plt.ylim(0, 1.05)
    plt.title("Rain Prediction: Model Comparison")
    plt.legend()
    plt.grid(True, axis="y")
    plt.tight_layout()


def predict_day(temp, hum, pres, wind, models, clf_scaler, km, km_scaler, remap, pi, A, B):
    day = pd.DataFrame([[temp, hum, pres, wind]], columns=[TEMP_COL, HUM_COL, PRES_COL, WIND_COL])
    cluster = remap[int(km.predict(km_scaler.transform(day))[0])]
    print("\nToday's weather type:", STATE_NAMES[cluster])
    day_s = clf_scaler.transform(day)
    for name, model in models.items():
        p = model.predict_proba(day_s)[0][1]
        print(f"{name}: rain tomorrow = {'YES' if p >= 0.5 else 'NO'} (probability {p:.2f})")


def main():
    df = load_data()
    print("\nData shape:", df.shape)
    print(df.describe().round(2))

    states, km, km_scaler, remap = find_hidden_states(df)
    print("\nDays in each weather type:")
    for s in range(N_STATES):
        print(STATE_NAMES[s], ":", int(np.sum(states == s)))

    obs = make_observations(df)
    pi, A, B = estimate_hmm(states, obs)
    print("\nInitial probabilities pi:\n", pi.round(3))
    print("\nTransition matrix A:\n", A.round(3))
    print("\nEmission matrix B:\n", B.round(3))

    window_ll, window_start, threshold, unusual, vit_acc, decoded = run_hmm(states, obs, A, B, pi)
    first_seq = obs[:WINDOW]
    ll = forward_log(first_seq, A, B, pi)
    _, path = viterbi_log(first_seq, A, B, pi)
    print("\nFirst week observations:", first_seq.tolist())
    print("Forward probability:", np.exp(ll))
    print("Viterbi weather sequence:", [STATE_NAMES[i] for i in path])
    print("Actual weather sequence :", [STATE_NAMES[i] for i in states[:WINDOW]])
    print("\nViterbi accuracy over all weeks: {:.2f}%".format(vit_acc * 100))
    print("Unusual weeks (start day, log-probability):", unusual)

    results, models, clf_scaler = rain_models(df)
    print("\nModel comparison:")
    print(results.round(3).to_string(index=False))
    best = results.loc[results["F1"].idxmax(), "Model"]
    print("\nBest model by F1-score:", best)

    mean_day = df[[TEMP_COL, HUM_COL, PRES_COL, WIND_COL]].mean()
    predict_day(mean_day[TEMP_COL], mean_day[HUM_COL], mean_day[PRES_COL], mean_day[WIND_COL],
                models, clf_scaler, km, km_scaler, remap, pi, A, B)
    rainy_day = df[states == 2][[TEMP_COL, HUM_COL, PRES_COL, WIND_COL]].mean()
    predict_day(rainy_day[TEMP_COL], rainy_day[HUM_COL], rainy_day[PRES_COL], rainy_day[WIND_COL],
                models, clf_scaler, km, km_scaler, remap, pi, A, B)

    plot_clusters(df, states)
    plot_matrices(A, B)
    plot_decoded(states, decoded)
    plot_likelihood(window_ll, threshold)
    plot_models(results)
    plt.show()
```

```
if __name__ == "__main__":
    main()
```

Start coding or generate with AI.